LAB CYCLE: 2 **DATE:18/09/2025**

PROGRAM NO: 10

K-Nearest Neighbor

Aim: To implement K-Nearest Neighbor algorithm using any dataset available in public domain and evaluate its accuracy.

Program:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn import metrics
iris=load_iris()
x=iris.data
y=iris.target
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.3,random_state=1)
c_knn=KNeighborsClassifier(n_neighbors=3)
c_knn.fit(x_train,y_train)
y_pred=c_knn.predict(x_test)
print("Accuracy:",metrics.accuracy_score(y_test,y_pred))
sample=[[2,2,3,3]]
pred=c_knn.predict(sample)
pred_v=[str(iris.target_names[p]) for p in pred]
print(pred_v)
```

Output:

```
Accuracy: 0.9777777777777777
['versicolor']
```

Result: The program was executed successfully and output verified.

LAB CYCLE: 2 **DATE:18/09/2025**

PROGRAM NO: 11

Naive Bayes

Aim: To implement Naive Bayes classification algorithm on the following data of a certain set of patients seen by a doctor and to check whether the doctor can conclude that a person having chills, fever, mild headache and without running nose has flu?

Chills	Running nose	Headache	Fever	Has flu
Y	N	mild	Y	N
Y	Y	no	N	Y
Y	N	strong	Y	Y
N	Y	mild	Y	Y
N	N	no	N	N
N	Y	strong	N	Y
N	Y	strong	N	N
Y	Y	mild	Y	Y

Program:

```
from sklearn.naive_bayes import CategoricalNB
from sklearn import preprocessing
# Data
chills = ['Y', 'Y', 'Y', 'N', 'N', 'N', 'N', 'Y']
running_nose = ['N', 'Y', 'N', 'Y', 'N', 'Y', 'Y', 'Y']
headache = ['mild', 'no', 'strong', 'mild', 'no', 'strong', 'strong', 'mild']
fever = ['Y', 'N', 'Y', 'Y', 'N', 'Y', 'N', 'Y']
has_flu = ['N', 'Y', 'Y', 'Y', 'N', 'Y', 'N', 'Y']
# Encode and train
le = preprocessing.LabelEncoder()
features = list(zip(le.fit_transform(chills),
```

```
le.fit_transform(running_nose),
le.fit_transform(headache),
                le.fit_transform(fever)))
label = le.fit_transform(has_flu)
model = CategoricalNB()
model.fit(features, label)
# Predict
predicted = model.predict([[1, 0, 0, 1]])  # Y, N, mild, Y
print("Prediction:", le.inverse_transform(predicted)[0])
print("Accuracy:", model.score(features, label))
```

Output:

```
Prediction: Y
Accuracy: 0.75
```

Result: The program was executed successfully and output verified.

LAB CYCLE: 2 **DATE:25/09/2025**

PROGRAM NO: 12

Decision Tree

Aim: To implement Decision Tree classification algorithm using any dataset available in public

Program:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score

# Load the dataset
data = load_iris()
print("Data shape:", data.data.shape)
print("Classes to predict:", data.target_names)
print("Features:", data.feature_names)

# Define features and target
X = data.data
y = data.target
print("Feature shape:", X.shape)
print("Target shape:", y.shape)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=50, test_size=0.25)
```

```
# -------------------- Model 1: Gini --------------------
clf_gini = DecisionTreeClassifier()
clf_gini.fit(X_train, y_train)
y_pred_gini = clf_gini.predict(X_test)

print("\n--- Using Gini ---")
print("Accuracy on train data:", accuracy_score(y_train, clf_gini.predict(X_train)))
print("Accuracy on test data:", accuracy_score(y_test, y_pred_gini))

# -------------------- Model 2: Entropy --------------------
clf_entropy = DecisionTreeClassifier(criterion='entropy')
clf_entropy.fit(X_train, y_train)
y_pred_entropy = clf_entropy.predict(X_test)

print("\n--- Using Entropy ---")
print("Accuracy on train data:", accuracy_score(y_train, clf_entropy.predict(X_train)))
print("Accuracy on test data:", accuracy_score(y_test, y_pred_entropy))

# -------------------- Model 3: Entropy + min_samples_split --------------------
clf_entropy_split = DecisionTreeClassifier(criterion='entropy', min_samples_split=50)
clf_entropy_split.fit(X_train, y_train)
y_pred_entropy_split = clf_entropy_split.predict(X_test)

print("\n--- Using Entropy with min_samples_split=50 ---")
print("Accuracy on train data:", accuracy_score(y_train, clf_entropy_split.predict(X_train)))
print("Accuracy on test data:", accuracy_score(y_test, y_pred_entropy_split))

# -------------------- Visualization of the tree --------------------
plt.figure(figsize=(12, 8))
plot_tree(clf_entropy,
```

```
        filled=True,
        feature_names=data.feature_names,
        class_names=data.target_names)
plt.title("Decision Tree (Entropy)")
plt.show()
```

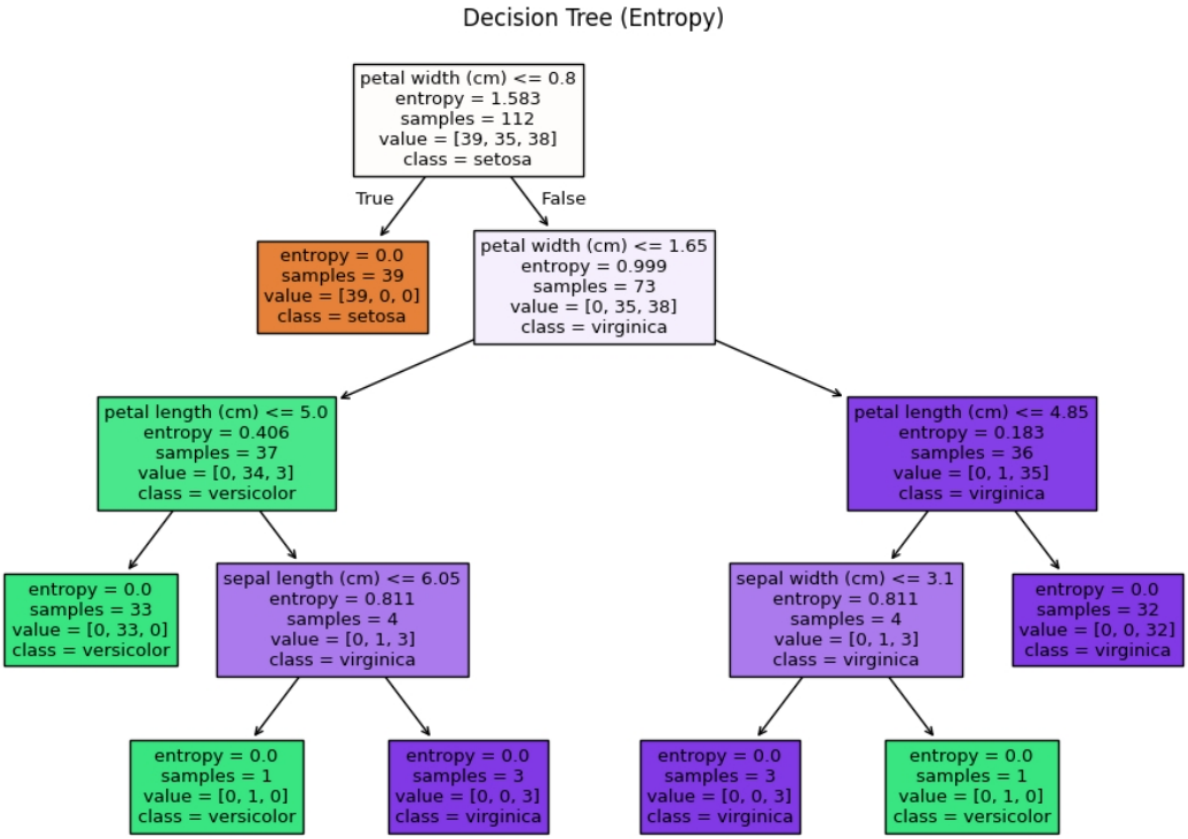
Output:

```
Data shape: (150, 4)
Classes to predict: ['setosa' 'versicolor' 'virginica']
Features: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Feature shape: (150, 4)
Target shape: (150,)

--- Using Gini ---
Accuracy on train data: 1.0
Accuracy on test data: 0.9473684210526315

--- Using Entropy ---
Accuracy on train data: 1.0
Accuracy on test data: 0.9473684210526315

--- Using Entropy with min_samples_split=50 ---
Accuracy on train data: 0.9642857142857143
Accuracy on test data: 0.9473684210526315
```



Result: The program was executed successfully and output verified.

LAB CYCLE: 2 **DATE:09/10/2025**

PROGRAM NO: 13

Linear Regression

Aim: To implement linear regression techniques using standard dataset available in public

Program:

```
#Load sklearn Libraries
#import libraries
import matplotlib.pyplot as plt
import numpy as np
from sklearn import datasets,linear_model
from sklearn.metrics import mean_squared_error, r2_score

#Load Data
#load diabetes dataset

diabetesX,diabetesY=datasets.load_diabetes(return_X_y=True)

#split datasets
diabetesX = diabetesX[:,np.newaxis,2]

#split the data into training/testing sets
diabetesXtrain=diabetesX[:-20]
diabetesXtest=diabetesX[-20:]

#split the targets into training/testing sets
diabetesYtrain=diabetesY[:-20]
diabetesYtest=diabetesY[-20:]
```

```
#Creating the model
#create linear regression object
regr=linear_model.LinearRegression()

#train the model using training sets
regr.fit(diabetesXtrain,diabetesYtrain)

#Make prediction
#make predictions using the testing set
diabetesYpred=regr.predict(diabetesXtest)

#finding coefficients and mean square error
#coefficients
print("Coefficients:\n",regr.coef_)

#mean squared error
print("Mean squared error:\n",mean_squared_error(diabetesYtest,diabetesYpred))

#coefficient of determination:1 is perfect prediction
print("Coefficient of determination:",r2_score(diabetesYtest,diabetesYpred))
plt.scatter(diabetesXtest,diabetesYtest,color='black')
plt.plot(diabetesXtest,diabetesYpred,color='blue',linewidth=3)
plt.xticks()
plt.yticks()
plt.show()
```

Output:

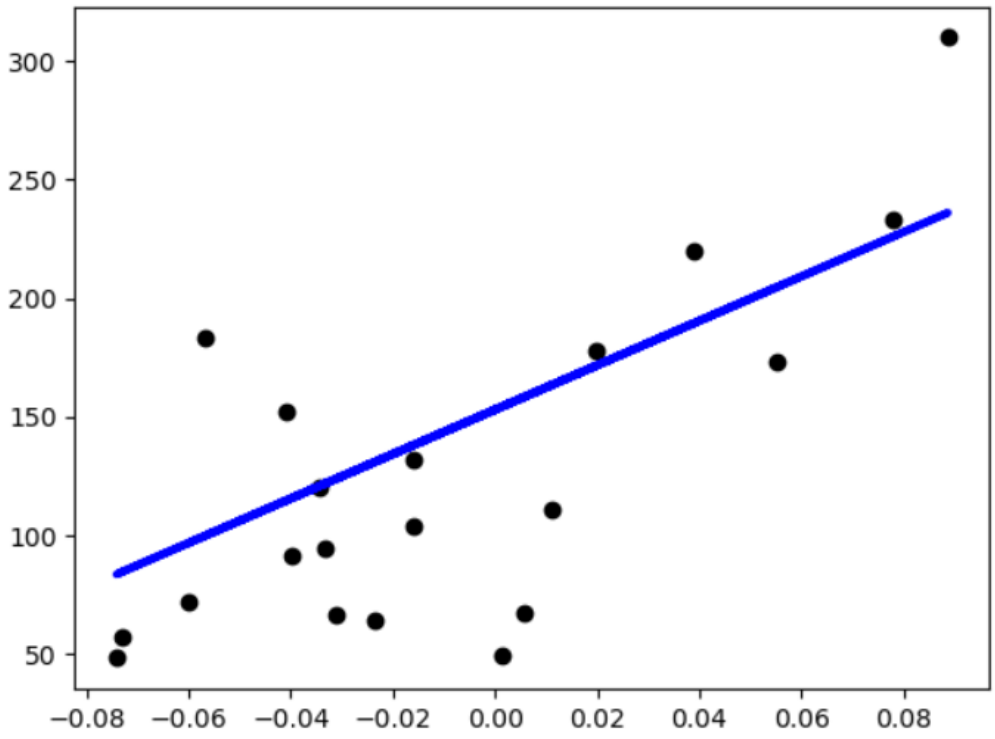
Coefficients:

[938.23786125]

Mean squared error:

2548.07239872597

Coefficient of determination: 0.47257544798227147



Result: The program was executed successfully and output verified.

LAB CYCLE: 2 **DATE:09/10/2025**

PROGRAM NO: 14

Support Vector Machine

Aim: To implement Support Vector Machine using standard dataset available in public and evaluate its accuracy.

Program:

```
from sklearn.model_selection import train_test_split
from sklearn import datasets
from sklearn import svm
from sklearn import metrics
cancer=datasets.load_breast_cancer()
x_train, x_test, y_train, y_test=train_test_split(cancer.data,cancer.target,test_size=0.3,
random_state=109)
clf=svm.SVC(kernel='linear')
clf.fit(x_train, y_train)
y_pred=clf.predict(x_test)
print("Actual values:", y_test)
print("Predicted values:", y_pred)
print("Accuracy:", metrics.accuracy_score(y_test, y_pred))
print("Precision:", metrics.precision_score(y_test, y_pred))
print("Recall:", metrics.recall_score(y_test, y_pred))
```

Output:

```
Actual values: [1 1 0 0 1 0 1 1 1 0 0 0 1 0 1 1 0 0 1 0 1 1 0 0 1 1 0 1 1 1 1 0 1 1 1 1 1
 0 1 1 0 1 0 1 1 1 0 1 0 0 1 1 0 1 1 0 1 1 1 0 0 1 0 1 0 0 1 1 1 1 0 1 1 1
 0 1 0 1 1 1 1 1 1 1 0 1 1 1 1 1 0 0 1 0 1 1 1 1 1 1 0 0 1 1 0 1 1 0 1 0 1 1
 0 1 1 0 0 0 1 0 1 1 1 1 1 1 0 1 0 1 0 1 0 0 0 0 0 0 1 1 0 1 1 1 0 1 1 0 0 1 0
 0 0 1 1 1 1 1 0 0 1 0 1 1 1 1 1 1 1 1 1 0 1 1 1]
Predicted values: [1 1 0 0 1 0 1 1 1 0 0 0 1 0 0 1 0 0 1 0 1 1 0 0 1 1 0 1 1 1 1 0 1 1 1 1 1
 0 1 1 0 1 0 1 1 1 1 1 0 0 1 1 0 1 1 0 1 1 1 0 0 1 0 1 0 0 1 1 1 1 0 1 1 1
 0 1 0 1 1 1 1 1 1 1 0 0 1 1 1 1 0 0 0 0 1 1 1 1 1 0 0 1 0 0 1 1 0 1 0 1 1
 0 1 1 0 0 0 1 0 1 1 1 1 1 1 0 1 0 1 0 1 0 0 0 1 0 1 1 0 1 1 1 0 1 1 0 0 1 0
 0 0 1 1 1 1 1 0 0 1 0 1 1 1 1 1 1 1 1 1 0 1 1 1]
Accuracy: 0.9649122807017544
Precision: 0.9811320754716981
Recall: 0.9629629629629629
```

Result: The program was executed successfully and output verified.